

How the prototype actually works

Every algorithm, constant and design decision in the build — written for the reviewer who wants to know what is really happening between the photograph and the sealing date, and for the person who has to answer them.

NINE PARTS · FIFTY QUESTIONS ANSWERED · EVERY FIGURE TAKEN FROM A REAL RUN

CONTENTS

- | | |
|--|-------------------------------|
| 1 The claim, in one page | 6 The priority formula |
| 2 Image processing: finding the crack | 7 Backend architecture |
| 3 GPS: reading it, writing it, not trusting it | 8 From this laptop to a truck |
| 4 Pixels into millimetres | 9 Fifty questions, answered |
| 5 Growth, lead time and the statistics | |

The claim, in one page

What the prototype does, and the single sentence everything else in this document exists to support.

Sealing a crack costs a fraction of patching the pothole it becomes, and the evidence for that has been settled since the 1990s. The reason Indian municipalities cannot act on it is not ignorance and not budget — it is **timing**. A crack photographed once is a defect with no date attached.

RAAHI photographs the same square metre of road on successive mornings, measures the crack each time, fits a growth rate through the measurements, and turns that rate into a date. Three things have to work for that to be true, and the rest of this document is those three things in detail:

- 1 Find and measure the crack in one photograph**
Classical image processing in the browser — threshold, morphological opening, connected components, and four shape gates that separate a crack from road speckle. Part two.
- 2 Recognise that it is the same crack tomorrow**
GPS is accurate to about ± 5 m and two cracks four metres apart are different cracks, so position alone cannot do it. Three independent signals vote. Part three.
- 3 Turn a sequence of measurements into a decision**
Least-squares fit, a confidence interval on the slope, a threshold crossing date, and a priority that accounts for the rain and the traffic the crack actually faces. Parts four to six.

The numbers this document keeps returning to

Everything below comes from one real run: eighteen photographs of one crack over seventeen days, uploaded through the browser, committed in the repository at `docs/sample-run/` with the input images beside the output JSON.

readings	18 over 17.01 days
measured	110.1 mm → 134.3 mm (+24.2 mm)
growth rate	1.392 ± 0.057 mm/day 95% confidence interval 1.271 to 1.513 mm/day
fit quality	R² 0.974 RMSE 1.18 mm largest single residual 2.57 mm
threshold	150 mm (adjustable)
crossing	11.3 days 10.4 to 12.4 days across the confidence interval
lead time	28.3 days first sighting to predicted failure
priority	$1.392 \times 2.17 \times 1.00 = 3.016 \rightarrow 77$ URGENT

THE SENTENCE TO DEFEND

Detecting a crack in one photograph is a solved problem and we claim no improvement on it. Measuring how fast the same crack is growing requires returning to the same metre of road for weeks, which no survey van, satellite or complaint app can do — and which a municipal waste fleet already does, every morning, for free.

Image processing: finding the crack

Seven stages, all running in the browser, with the exact constants and the reason each one is there.

2.1 Why classical vision and not a neural network

This will be the first question, so it is answered first. The prototype uses thresholding, morphology and connected components — **not** a trained model. That is a deliberate choice for this stage, and the reasons are specific:

- **The measurement, not the detection, is the contribution.** A model would give a bounding box. This project needs a *traced length in millimetres*, which a box does not provide.
- **It is auditable.** Every step below can be checked by hand on a single photograph. A reviewer can disagree with a number and see exactly which line produced it. A network's failure is a black box.
- **It runs anywhere with nothing installed.** The whole prototype is Python's standard library plus browser JavaScript. No CUDA, no PyTorch, no model weights to ship.
- **It does not need training data we have not earned.** Quoting a mAP from someone else's paper is not a result.

The production path is a TFLite detector fine-tuned on RDD2022 running on the phone, with this geometry stage taking its output. Part eight describes that handover. The stages below do not change — they run on a cropped region instead of a whole frame.

2.2 The pipeline

- 1 Downscale to a working width of 760 px**

A 12-megapixel phone photograph is 4000 px wide. Processing it whole costs sixteen times the work for no gain: a crack is a large feature and survives the reduction, while sensor noise — which is per-pixel — is averaged away by the browser's area-based resampling. 760 px is wide enough that a 3 mm crack at 1.2 m is still two to three pixels across.

Everything downstream, including the scale calculation, uses this width. That consistency matters: change the working width and the millimetres change with it, which is why the width is stored with every reading.

2

Convert to greyscale, ITU-R BT.601 weights

$$\text{gray} = 0.299 \cdot R + 0.587 \cdot G + 0.114 \cdot B$$

These are the luminance coefficients: the eye is most sensitive to green, least to blue. A plain average of the channels would let a blue-tinted shadow read as darker than it looks, and shadows are the main false positive on a road.

The mean of the whole greyscale frame is kept as `sceneMean`. It is used in stage six to measure how much darker a candidate is than the road around it.

3

Threshold — swept, not fixed

A pixel is marked when `gray < threshold`. But the correct threshold on wet tarmac at 7 a.m. is not the correct threshold on dry tarmac at 9 a.m., so a fixed number is useless in the field.

The app sweeps **threshold 70 to 180 in steps of 10** — twelve full passes of the pipeline — and keeps the one whose winning candidate has the highest *merit*:

$$\text{merit} = \text{contrast} \times (\text{span} \div \text{image diagonal})$$

Contrast times reach. Raising a threshold always finds more dark pixels, so the naive "most pixels found" criterion returns the whole photograph at threshold 180. Merit does not: a huge dark blob has near-zero contrast against the scene mean, and scores badly. Only settings that leave something *crack-shaped* behind are considered at all.

4

Morphological opening — dilate 3×3, then erode 3×3

An asphalt surface is aggregate: thousands of small dark specks that cross any threshold a crack crosses. Opening removes them.

Dilate sets a pixel if any of its 3×3 neighbours is set. Isolated specks grow slightly; the crack, being continuous, also grows and its small gaps close. **Erode** then clears a pixel unless all of its 3×3 neighbourhood is set. Specks — now isolated blobs a few pixels across — vanish entirely. The crack, which is long and connected, survives at close to its original width.

The order matters. Erode-then-dilate (a *closing*) would do the opposite: fill the gaps and keep the speckle.

5

Connected components, 8-connectivity, iterative flood fill

Every surviving pixel is assigned to a blob. The fill is 8-connected — diagonal neighbours count — because a crack running at 30° to the pixel grid is a staircase of diagonal steps, and 4-connectivity would chop it into fragments.

It is written with an explicit `Int32Array` stack rather than recursion. A crack can span a thousand pixels and recursive flood fill would overflow the JavaScript call stack on a large photograph.

For each blob the pass records area, bounding box, centroid, and the pixel list. Blobs under **180 px** are discarded immediately.

6

Score every candidate on four shape properties

This is where a crack is separated from everything else dark in the frame. Four gates, described in the table below, and a ranking score.

7

Measure the winner: $\text{arc length} = \text{perimeter} \div 2$

A boundary pixel is one on the frame edge or with at least one unset 4-neighbour. Counting them gives the perimeter. For a long thin shape the perimeter is approximately twice the length — once up one side, once back down the other — so half the perimeter is the traced length *along the crack*, following every bend.

This is the key measurement, and it is why the app reports a longer figure than a straight-line ruler would: a crack that wanders is longer than the distance between its endpoints.

2.3 The four gates, exactly

A candidate is marked `plausible` only if it passes all four. Failing does not delete it — it drops below the plausible ones in the ranking and is still reachable through *Wrong shape? Next candidate*.

GATE	TEST	WHAT IT REJECTS
Not too big	$\text{area} / (W \cdot H) < 0.06$	A whole shadowed half of the frame. A crack is a line, not a region — it never covers 6% of a photograph.
Not too tangled	$\text{arcLength} / \text{span} < 4.5$	Scattered speckle that survived the opening. A real crack wanders, but its traced length is never more than four and a half times the straight-line distance across it.
Long enough	$\text{span} > 0.06 \cdot \text{diagonal}$	A stone, a leaf, a bolt head. Anything that does not reach across a meaningful fraction of the frame.
Solid enough	$\text{area} > 220 \text{ px}$	A thin scratch on the lens or a two-pixel artefact.

THE RANKING SCORE

$$\text{score} = (P^2 / A) \times 1/(1 + 3.2 \cdot \text{offset}) \times (0.18 \text{ if it touches the border})$$

P = perimeter, A = area, offset = distance of the centroid from frame centre ÷ diagonal

P^2/A is thinness. For a circle it is $4\pi \approx 12.6$, its minimum. For a long thin line it is large. It is the classical isoperimetric measure and it is scale-invariant, which matters because the same crack photographed from 1 m and from 1.5 m must score alike.

The centre bias reflects how somebody photographs a crack: they point at it. A blob at the edge of the frame is more likely to be a kerb or a shadow.

The border penalty ($\times 0.18$) is the strongest single term. A shape running off the edge of the frame has been cut by the frame, so its length is unknowable — and measuring an unknowable length is worse than measuring the wrong thing.

FINAL ORDERING

```
sort by: plausible first
         then contrast × span      (dark and long beats faint and long)
         then score                (thinness, with the centre and border terms)
```

Contrast is the mean darkness of the blob measured against the scene mean, normalised: $(\text{sceneMean} - \text{meanOfBlob}) / 255$. A crack is a hole in the surface and reads far darker than the road. Speckle that only just crossed the threshold barely differs from its surroundings, and this term demotes it.

2.4 A worked example

Day one of the committed run, 900×640 px:

source	900 × 640 px
working size	760 × 540 px (scale 0.844)
threshold chosen	by sweep, on merit
blob area	≈ 400 px after opening
perimeter	≈ 310 px
arc length	155 px (perimeter ÷ 2)
scale	0.7105 mm/px (see part four)
reported	110.1 mm

2.5 What breaks it — stated, not discovered

CONDITION	EFFECT	HANDLING
Hard shadow across the crack	Shadow edge may be darker than the crack	contrast × span demotes it; the operator can cycle candidates
Wet road	Whole surface darkens; contrast falls	threshold sweep adapts; the scene fingerprint flags the change
Painted lane marking	A long thin bright shape	never crosses a darkness threshold – bright, not dark
Tar patch / crack sealant	Long, thin, very dark	genuine false positive. See the persistence rule, part eight
Motion blur	Crack smears; area rises, contrast falls	usually fails the tangle or size gate and is reported as implausible
Night / headlights	Specular highlights, deep shadow	out of scope. Daylight only, stated on the overview tab

THE HONEST WEAKNESS

A **tar patch or an old sealant line** is long, thin and very dark. Geometry alone cannot distinguish it from a crack, and the prototype does not. The answer is not a better shape test — it is *time*: a sealant line is present on day one and never grows, while a crack appears and lengthens. That rule needs the repeat-visit data this project is built to collect, and it is described in part eight.

GPS: reading it, writing it, and not trusting it alone

Where the coordinates come from, how they are parsed byte by byte, and why the whole project would fail if it relied on them.

3.1 What is actually in the file

A camera with location enabled writes the fix into the image file itself, in a structure called **EXIF** — an embedded TIFF directory. It is not a sidecar file and not a database entry: it is inside the photograph.

These are the tags RAAHI reads, with real values from day one of the committed run:

EXIF TAG	HEX	VALUE	MEANING
GPSLatitude	0x0002	28.613969° N	Three rationals: degrees, minutes, seconds
GPSLongitude	0x0004	77.209064° E	Same form; the ref tag carries N/S and E/W
GPSAltitude	0x0006	216.0 m	With a ref byte for above/below sea level
GPSImgDirection	0x0011	98.68°	Which way the camera pointed. Central to matching
GPSHPositioningError	0x001F	± 4.5 m	The device's own estimate of its error
GPSTimeStamp / DateStamp	0x0007 / 0x001D	06:52:00 UTC	Satellite clock, always UTC
DateTimeOriginal	0x9003	2026-08-21 06:52	Shutter time. This dates the reading
FocalLengthIn35mmFilm	0xA405	80 mm	The lens. This is what makes millimetres possible

3.2 How the parser works

`raahi_backend/exif.py` is a hand-written TIFF reader — about 260 lines, no dependencies. The walk is:

- 1 Locate the TIFF header inside the container
 - JPEG walk segment by segment to the APP1 marker (0xFFE1) tagged "Exif\0\0"
 - PNG walk chunk by chunk to the eXIf chunk
 - HEIC scan for the "Exif\0\0" signature
- 2 Read the byte order: "II" little-endian or "MM" big-endian, then magic 42
- 3 Read IFD0 – the main directory. Each entry is 12 bytes:
 - tag (2) | format (2) | component count (4) | value or offset (4)
 - Values of 4 bytes or fewer sit inline; longer ones are an offset into the block
- 4 Follow the pointers: 0x8769 to the Exif sub-IFD, 0x8825 to the GPS sub-IFD
- 5 Convert
 - degrees/minutes/seconds + N/S/E/W -> signed decimal degrees
 - "2026:08:21 06:52:00" -> ISO-8601

WHY WRITE A PARSER RATHER THAN IMPORT ONE

The whole app must run with `python3 raahi.py` and nothing installed — no pip, no internet, on a municipal laptop. That constraint is worth about 260 lines. A malformed file returns an empty result rather than raising: a photograph with no GPS is a normal case, not an error.

3.3 Writing GPS into a photograph

A frame that arrives with no metadata — anything through a chat app, a screenshot, a webcam capture — carries no position. Rather than keep the coordinates only in the database, `geotag.py` writes them *into the file* as it is stored: it builds a complete little-endian TIFF block (IFD0 → Exif IFD + GPS IFD, offsets computed in a layout pass before the bytes are emitted) and inserts it as a JPEG APP1 segment or a PNG eXIf chunk.

Afterwards the photograph carries its own coordinates. Copy it anywhere, open it in any EXIF viewer, and they are there — the record and the pixels cannot drift apart, which is what matters if a repair bill is ever disputed.

A photo that arrived with its own GPS is never rewritten. The camera's word beats ours, and the interface says which of the two happened.

3.4 Precision is not accuracy

1e-6° of latitude ≈ 11 cm · stated error ± 4.5 m

The coordinate is written to centimetre precision and known to about five metres. The app shows both, always.

Six decimals looks authoritative and is not. Consumer GPS under open sky is good to roughly 3–5 m; near buildings, multipath pushes it to 10 m or worse. This is the single most important limitation in the project, and the design assumes it rather than hoping about it.

3.5 Three signals, because one is not enough

Two cracks four metres apart are different cracks. A ± 5 m fix cannot separate them. So the revisit decision is a vote:

SIGNAL	COMPUTATION	TOLERANCE	RULES OUT
Distance	haversine, $R = 6\,371\,008.8$ m	15 m	A different street
Heading	smallest circular difference	45°	Standing still, photographing the opposite kerb
Appearance	64-bit difference hash	≤ 12 of 64 bits	Everything else — and works with no GPS at all

same spot $\iff$ (distance ≤ 15 m $\wedge$ heading $\leq 45^\circ$) $\vee$ (hash ≤ 7 bits $\wedge$ position not contradicting)

Two independent routes. The second is what lets a photograph with no GPS at all still find its own history.

Why 15 m and not 5 m. The tolerance has to exceed the error of the measurement, or genuine revisits are rejected. Two fixes each good to ± 5 m can differ by 10 m while describing the same point. 15 m accepts that; the heading and appearance terms carry the discrimination that the loosened radius gives up.

Why heading matters more than it looks. A person standing on a kerb can photograph the road to their left or the footpath to their right without moving. Position is identical; the crack is not. `GPSTimeDirection` separates the two cases and nothing else in the sensor set can.

3.6 The scene fingerprint, in detail

A difference hash. The browser sends a 32×32 luminance grid; the server box-averages it to 9×8 and emits one bit per horizontal neighbour pair:

```
for each of 8 rows:
  for each of 8 columns:
    bit = 1 if cell[row][col] > cell[row][col+1] else 0
→ 64 bits, stored as 16 hex characters
```

The comparison is **Hamming distance** — how many of the 64 bits differ.

WHY A DIFFERENCE HASH AND NOT AN AVERAGE HASH

An average hash compares each cell to the image mean, so a uniform brightness shift flips many bits at once — and brightness is exactly what changes between a 7 a.m. and a 9 a.m. pass. A difference hash compares *neighbour to neighbour*. Add 20 to every pixel and every comparison gives the same answer. It is invariant to exposure and to linear contrast change, while remaining sensitive to the layout of the scene.

Measured: across the eighteen demo passes, deliberately rendered at exposures spanning 28 grey levels, the fingerprint never differed from day one by more than **2 bits of 64** — against a matching gate of 12.

3.7 The two thresholds

12 of 64 means the scene matches. **7 of 64** is "unmistakable" — strong enough to carry the decision on its own when there is no GPS at all. 64 bits gives 2^{64} possible hashes; agreeing within 7 by chance is vanishingly unlikely for unrelated road scenes, while leaving room for rain, a leaf, a parked wheel.

3.8 When the signals disagree

Position matching while appearance does not is **reported, not silently accepted**:

SHOWN ON SCREEN

"The position matches but the scene does not. If this is a different crack nearby, set the spot by hand before saving."

Usually it is the light or a wet road. Sometimes it is a different crack two metres over. The system does not know which, so it says so and offers the override. Every match, agreeing or not, prints its own reasoning:

1.2 m from the recorded position; camera pointing within 4 deg of before; scene matches (1/64 bits differ)

3.9 The anchor improves with use

After every pass the spot is re-centred on the arithmetic mean of its own fixes, and its heading on the *circular* mean — 350° and 10° average to 0° , not to 180° . With n independent fixes the standard error of the mean falls as $1/\sqrt{n}$, so the recorded position of an eighteen-pass spot is roughly four times better than a single fix. Tomorrow's match is tighter than today's.

Pixels into millimetres

The optics, the four-rung fallback ladder, and the argument that makes the whole measurement defensible.

4.1 The pinhole model

A 35 mm-equivalent focal length is defined against a 36 mm-wide frame. By similar triangles, a lens of equivalent focal length f at distance d covers a ground width of $36 \cdot d/f$. Divide by the pixel width and one pixel is:

$$\text{mm per pixel} = (36 \times \text{distance}_{\text{mm}}) \div (\text{focal}_{35\text{mm}} \times \text{image width}_{\text{px}})$$

No reference object in the frame. No ruler. Just the lens and the shooting distance.

Worked, for the committed run:

focal 80 mm equivalent, distance 1200 mm, working width 760 px

$$\frac{36 \times 1200}{80 \times 760} = \frac{43\,200}{60\,800} = 0.7105 \text{ mm per pixel}$$

day 1: 155 px $\times$ 0.7105 = 110.1 mm ✓ matches the stored reading

4.2 The ladder

Four sources, best first. **Every reading records which rung it used** and the interface prints it.

#	SOURCE	ACCURACY	WHEN
1	Calibration at this spot ruler measured at that location	best	If somebody has calibrated there
2	Global calibration rescaled if the photo is a different width	good	Same camera and distance as the calibration
3	Lens optics focal length from EXIF × stated distance	fair	Any phone photograph with metadata intact
4	Assumed 26 mm phone lens labelled <i>assumed</i> on every reading	approximate	Metadata stripped — a chat app, a screenshot

WHY RUNG FOUR EXISTS AT ALL

Refusing to measure was worse than it sounds. A user's first upload of a WhatsApp-forwarded photograph produced no scale, so no growth curve, so a blank screen and an instruction to go and calibrate before they had seen the app do anything. Rung four gives a number and marks it *assumed*. The next paragraph is why that is defensible.

4.3 The argument that carries the whole measurement

This is the answer to "how do we trust your millimetres?", and it is worth learning verbatim.

Tracing a crack's outline adds the outline's own width to the length. Call that bias b . It is roughly constant for a given camera, distance and threshold. So each reading is:

$$L_{\text{measured}} = L_{\text{true}} + b$$

Absolute length therefore carries an error. But the growth between two passes is:

$$\Delta = (L_2 + b) - (L_1 + b) = L_2 - L_1$$

The bias cancels exactly.

The same holds for the slope of a least-squares fit: adding a constant to every y shifts the intercept and leaves the gradient untouched. So **the growth rate — the number this project sells — is immune to a constant scale bias, while absolute length is not.**

The condition is that the bias be the *same* across passes, which requires a consistent camera, distance and detector. The prototype meets that by construction: one operator, one phone, one threshold policy. The truck deployment meets it better, with the camera on a rigid mount at a fixed height.

4.4 Error budget

SOURCE	MAGNITUDE	CANCELS IN GROWTH?
Outline width bias	~1–3 mm	Yes — constant
Wrong assumed focal length	up to $\pm 30\%$ of scale	Scales both readings alike; the <i>rate</i> is proportionally wrong but the trend and the date ordering survive
Shooting distance varies pass to pass	± 10 cm at 1.2 m $\approx \pm 8\%$	No — this is the real limit on rate accuracy
Threshold choice differing between passes	± 1 –2 px on the trace	Partly. It shows as scatter about the fit
Detector picking a different candidate	large	Visible as an outlier; removable from the gallery

The measured scatter on the committed run — **RMSE 1.18 mm** about the fitted line, largest residual 2.57 mm — is the sum of the last three rows. The fixed-mount truck deployment removes the distance variation, which is the dominant term.

Growth, lead time and the statistics

How a sequence of measurements becomes a date, and how confident that date actually is.

5.1 Ordinary least squares, written out

Readings are (day, length) pairs, day counted from the first sighting. The fit minimises the sum of squared vertical residuals:

$$\text{slope} = \Sigma(x_i - \bar{x})(y_i - \bar{y}) / \Sigma(x_i - \bar{x})^2$$

$$\text{intercept} = \bar{y} - \text{slope} \cdot \bar{x}$$

Implemented in `growth.py` in about fifteen lines, standard library only. Two guards matter:

- **Fewer than two readings** → no rate. A single photograph is a defect, not a trend.
- $\Sigma(x - \bar{x})^2 \approx 0$ → every reading on the same day. The slope would divide by zero, so the app returns **SAME DAY** and asks for another morning.

5.2 R^2 , and why it is printed next to the answer

$$R^2 = 1 - SS_{\text{residual}} / SS_{\text{total}}$$

The share of the variation in length that the straight line explains.

The committed run gives $R^2 = 0.974$. Below 0.6 the app prints a warning on the face of the result and calls the date rough.

THE TRAP, AND WHY THE APP AVOIDS IT

Two points always fit a line perfectly — $R^2 = 1.000$ — and that fact must never be allowed to look like confidence. The app therefore always shows the *reading count* beside R^2 . " R^2 1.00 over 2 readings" reads very differently from " R^2 0.97 over 18", and it should.

5.3 How confident is 1.392 mm/day?

A reviewer who knows regression will ask for an interval, not a point. Computed from the committed run's eighteen residuals:

$$SE(\text{slope}) = \sqrt{SSE / (n-2) / \sum (x_i - \bar{x})^2} = 0.0570 \text{ mm/day}$$

n = 18, so 16 degrees of freedom; t = 2.12 at 95%

growth rate	1.392 mm/day
standard error	0.057 mm/day
95% interval	1.271 to 1.513 mm/day
RMSE	1.18 mm typical distance of a reading from the line
days to 150 mm	11.3 days 10.4 days at the fast end of the interval, 12.4 at the slow end

The crossing date is certain to within about ± 1 day at 95% confidence. That is the strongest single statistic in this project, and it is worth saying out loud: for a maintenance decision measured in fortnights, a ± 1 day uncertainty is far inside what the decision needs.

5.4 Lead time

$$\text{lead time} = \text{predicted crossing date} - \text{first sighting date}$$

28.3 days on the committed run

It exists only where the same square metre has been photographed more than once, which is why no Indian road authority publishes it. The fleet summary reports the mean, median, minimum and maximum across spots — and counts *only* spots where a crossing is actually dated, so the average cannot be flattered by cracks that never grew.

5.5 The verdict ladder

VERDICT	CONDITION	SIGNAL
SEAL NOW	latest $\geq$ threshold	red
SEAL SOON	crossing $\leq$ 14 days away	red
MONITOR	15–45 days	amber
WATCH	> 45 days	amber
STABLE	slope $\leq$ 0	green
BASELINE ONLY	one reading	grey
SAME DAY	all readings on one date	grey
NO SCALE	photographs but no millimetres	grey

GREY IS NEVER GREEN

A spot photographed once shows **grey**, not green. "We never went back" is not the same claim as "we checked and it is fine". Every road-condition dataset that reports coverage without recency makes exactly that conflation by omission. This one refuses to.

The priority formula

One measured term, two stated multipliers, and every constant returned in the API so nobody has to read the source to audit it.

$$\text{priority} = \text{growth rate} \times \text{rainfall factor} \times \text{traffic factor}$$

measured mm/day · dimensionless · dimensionless

the product has units of mm/day: the rate this crack would have with the rain and the traffic it actually faces

6.1 Rainfall

$$\text{factor} = 1 + (\text{expected mm over 14 days} / 60), \text{ capped at } 5.0$$

Water is the damage mechanism: it enters the crack, reaches the base course, the base loses support, and the surface fails under the next axle. A dry fortnight gives exactly 1.0, so a dry-season priority is growth × traffic and nothing invented.

The figure comes from a bundled table of **47 Indian stations**, monthly normals, apportioned across the days in the window and picked by nearest station with the distance reported. The cap exists because a 900 mm fortnight does not make a crack fifteen times more urgent — past a point the crack is simply saturated.

SAY THIS BEFORE YOU ARE ASKED

A **normal is not a forecast**. It says what a fortnight like this usually brings at that place. The API labels it basis: "normal" and never uses the word forecast. `POST /api/rainfall` takes a real IMD district figure and it is used instead, immediately.

6.2 Traffic

$$\text{factor} = \sqrt{(\text{commercial vehicles per day} / 300)}, \text{ clamped } 0.5 - 4.0$$

Commercial vehicles, not total AADT, because it is axle load rather than vehicle count that breaks a pavement. A residential lane — 300 cvpd, the reference — is exactly 1.0.

THE SQUARE ROOT IS THE ARGUABLE CHOICE, SO ARGUE IT FIRST

Pavement design uses a **fourth-power law** for structural damage from axle load. Applied here it would put a national highway 200× above a residential lane and drown the other two terms entirely — the ranking would become a road-class sort with a growth rate attached for decoration.

What is being ranked is *how much sooner a crack that is already growing needs sealing*, not how much total structural damage the road takes. A gentler curve keeps the measured term in charge. The exponent is an assumption, stated as one, and it is the first thing to re-fit against a season of real readings.

6.3 The score

$$\text{score} = 100 \times \log_{10}(\text{effective} / 0.05) / \log_{10}(10 / 0.05)$$

clamped to 0–100; 0.05 mm/day effective scores 0, 10 mm/day scores 100

EFFECTIVE MM/DAY	SCORE	BAND	ACTION
0.10	13	LOW	Keep photographing it
0.30	34	MEDIUM	Next ward round
0.50	43	HIGH	This fortnight's list
2.20	71	URGENT	Before the next rain
3.02	77	URGENT	← the committed run
10.0	100	URGENT	A pothole forming

Why logarithmic. The product spans orders of magnitude — a hairline at 0.05 mm/day on a dry lane against 1.5 mm/day on a monsoon arterial, which is another ten times on top. A linear scale saturates: everything worth looking at pins to 100 and the ranking stops ranking. This was found by building it linear first and watching four different spots all score 100.

6.4 What the monsoon costs

The rain term never alters the measured growth. `mm_per_day` stays what the readings said. What changes is a *second* crossing date computed at the effective rate, printed beside the dry-weather one:

dry weather	18 September
with the rain	12 September
difference	6 days what sealing before the rain is worth, as a date rather than a slogan

6.5 Every constant, and its status

CONSTANT	VALUE	STATUS
Rainfall reference	60 mm / fortnight	chosen
Rainfall cap	5.0	chosen
Traffic reference	300 cvpd	chosen
Traffic exponent	0.5	chosen
Traffic clamp	0.5 – 4.0	chosen
Score floor / ceiling	0.05 / 10.0 mm/day	chosen
Seal threshold	150 mm	user-set, per ward policy
Growth rate	–	measured

GET /api/risk returns all of these under assumptions , in the app's own output. Volunteering that they are chosen is worth more than any answer given after being caught.

Backend architecture

What happens to one photograph between the file picker and the database, and why the system is built the way it is.

7.1 The split: browser measures, server decides

Pixels stay in the browser. What the server receives is small: an arc length, a span, the working width, a 32×32 luminance grid, and the original file bytes for storage.

The division is deliberate. Anything that has to *survive a closed tab* — where the photo was taken, whether it is yesterday's crack, how fast it is growing — is the server's job. Anything that is pure computation on pixels already in the browser stays there, because sending 12 megapixels to a server to count dark pixels is a waste of a depot's bandwidth.

7.2 One upload, end to end

BROWSER

- 1 read file FileReader → data URL (original bytes, EXIF intact)
- 2 detect threshold sweep, opening, components, gates
- 3 fingerprint 32×32 luminance grid
- 4 POST /api/observations

SERVER (raahi_backend/api.py :: post_observation)

- 5 decode base64, reject over 24 MB
- 6 exif.read(bytes) GPS, heading, focal length, shutter time
- 7 resolve position EXIF → browser geolocation → manual, in that order
- 8 perceptual_hash() the 32×32 grid → 64-bit dHash
- 9 match_site() distance + heading + appearance vote
 - └ matched → this spot └ no match → create a new spot
- 10 settle the date EXIF clock → day number → upload time
- 11 geotag if needed write GPS into the file bytes (only if it had none)
- 12 save_photo() SHA-256 of the bytes → data/photos/<sha>.ext
- 13 _scale_for() the four-rung ladder → mm per pixel
- 14 length_mm = arc_px × mm_per_px
- 15 INSERT INTO observations
- 16 refresh_site_anchor() re-centre the spot on the mean of its fixes
- 17 growth.analyse() slope, R^2 , crossing date, verdict
- 18 risk.assess() growth × rain × traffic, score, band
- 19 return the lot as JSON

BROWSER

- 20 render position panel, growth figures, gallery

STEP 10 BEFORE STEP 11 — A BUG WORTH KNOWING ABOUT

These were originally the other way round. Geotagging wrote `DateTimeOriginal` into the file, and the date logic then read that stamp back and treated it as the camera's word — so a folder of eighteen undated photographs all landed on the same timestamp and produced no growth rate at all. The date is now settled from the original metadata *before* anything is written into it.

7.3 Data model

SQLite, five tables. Two carry the science:

TABLE	HOLDS	KEY COLUMNS
sites	A place on the road	lat, lon, direction_deg, phash, threshold_mm, road_class, commercial_vehicles_per_day, rain_mm_override, ward
observations	One photograph of one place	site_id, captured_at, lat/lon/altitude/accuracy, gps_source, phash, length_mm, arc_px, image_width, mm_per_px, scale_source, photo_sha, day_index, geotag_written, match_json
calibration	mm-per-pixel by scope	scope ('global' or a site id), mm_per_px, image_width
trials	Accuracy against a ruler	drew_mm, said_mm, error_mm
evals	Detector scores	split, model, payload

Three schema decisions worth defending:

- **scale_source** is stored on every row. Not just the number but where it came from. A reading measured by calibration and one measured by an assumed lens are different kinds of fact, and the database records which.
- **match_json** stores the full reasoning — the metres, the degrees, the bits — for every match. The report can reproduce, months later, exactly why a photograph was filed where it was.
- **Migrations run on start.** Columns added after release are applied with `ALTER TABLE` guarded by `PRAGMA table_info`, so an existing database on somebody's laptop survives an upgrade instead of needing a wipe.

7.4 Photographs are content-addressed

A photograph is stored as `data/photos/<sha256>.ext`. Consequences:

- Uploading the same file twice costs no extra disk.

- A reading traces back to the exact bytes it came from — a hash chain, and the basis for the signed, tamper-evident records the deck proposes for disputes.
- Lookup is O(1) by name; the folder is never listed.

7.5 Why SQLite and why the standard library only

The deck's production stack is PostgreSQL + PostGIS + FastAPI + Celery. This prototype is deliberately not that.

- **It has to run on a municipal laptop with no internet and no admin rights.** `python3 raahi.py` and nothing else. A dependency list is a demo that fails in the room.
- **The whole record is one file** a team-mate can copy, e-mail or delete.
- **SQLite is ACID and handles this volume trivially.** Fifty-four readings is nothing; the same schema migrates to PostGIS unchanged when spatial indexing over a city's worth of spots starts to matter.

7.6 The API

METHOD	PATH	RETURNS / DOES
GET	/api/state	Counts, calibration, fleet lead time
GET	/api/sites · /api/sites/<id>	Spots, with growth reports
GET	/api/observations	Every photograph, flat, for the gallery
GET	/api/schedule	The ranked seal list, every term shown
GET	/api/risk	The formula's working, terms and assumptions
GET	/api/report/<id>	Detect, track, predict, schedule, verify
GET	/api/rainfall?lat=&lon=	Expected rain and its basis
POST	/api/observations	A photo plus what the browser measured
POST	/api/sites/<id>/context	Road class, vehicle count, ward
POST	/api/rainfall	A real forecast, or a CSV of normals
DELETE	/api/observations/<id>	Remove one photograph; everything recomputes
DELETE	/api/records	Clear the round; keep calibration

7.7 Testing

`tools/selftest.py` starts a real server, writes real geotagged photographs, uploads them over HTTP and asserts **68** things about what comes back. Nothing is mocked — the detector inside the test is the browser's own, reimplemented in Python, so the figures it checks are the figures the app produces.

A sample of what it proves: GPS is read from the file; later passes land on the same spot and a photo 12 km away does not; growth $\times$ rain $\times$ traffic really multiplies out to the effective rate printed beside it; a supplied forecast displaces the bundled normals; a photograph sent with no metadata comes back off disk carrying the coordinates the app wrote into it; a deleted spot's id is never reissued.

From this laptop to a truck

What the production system does differently, and — more usefully — what it does not.

8.1 What survives unchanged

The mathematics is the contribution and none of it changes: the revisit vote, the pinhole scale, the least-squares fit with its confidence interval, the lead time, the priority formula, the honesty rules. Those are the parts a reviewer should test, because they are the parts that will still be there in the deployed system.

8.2 What changes

STAGE	PROTOTYPE	ON THE TRUCK
Trigger	A person chooses files	1–2 FPS, gated on speed 10–30 km/h, daylight and sharpness
Detection	Threshold + morphology in the browser	TFLite detector fine-tuned on RDD2022, on-device
Geometry	Assumes camera roughly overhead	Inverse perspective mapping to a metric top-down view
Re-localisation	GPS + heading + 64-bit scene hash	Plus ORB features and homography to the stored reference frame
Privacy	Not required — no people in frame	Face and plate blurring before anything is written to storage
Transport	localhost	~10 KB vector per detection, over depot Wi-Fi at night
Storage	SQLite + photo files	PostgreSQL + PostGIS, map-matched to OSM

8.3 Why a garbage truck

REQUIREMENT	WHY THE WASTE FLEET SATISFIES IT
Low motion blur	10–25 km/h — the slowest fleet a city owns
Usable light	Morning collection round
Complete coverage	Every residential lane, not only arterials
Daily repetition	Same route, same time, every day — this is the whole idea
Sync point	Returns to a depot each night; free Wi-Fi
One decision-maker	A Municipal Commissioner, a single pilot

8.4 The motion blur arithmetic

$$25 \text{ km/h} \approx 6.9 \text{ m/s} \times 1/1000 \text{ s shutter} = 6.9 \text{ mm of travel}$$

At a ground resolution near 1 mm/px that is a few pixels of smear along the direction of travel — survivable for a feature measured along its length.

This is precisely why the slow fleet was chosen and not a bus or a survey van. At 60 km/h the same shutter gives 17 mm of smear and the measurement stops meaning anything.

8.5 Bandwidth

A detection travels as a polyline and a class — roughly 10 KB — not as an image. A truck finding 200 defects a day sends about 2 MB, over depot Wi-Fi, once. There is no mobile data cost and no continuous video anywhere in the design.

8.6 The rule that separates infrastructure from damage

THE BEST IDEA IN THE PROJECT, AND IT NEEDS NO TRAINING DATA

A speed bump, a manhole, a lane marking and a tar patch appear in every pass since day one and never change. A crack appears one day and then grows.

That single rule separates permanent infrastructure from developing damage with no labelled dataset and no manual annotation whatsoever — and it is available only to a system that returns to the same metre every morning. It is also the answer to the tar-patch false positive in part two, and the reason the coverage claim and the accuracy claim are the same claim.

8.7 Privacy and the DPDP Act 2023

- Faces and number plates blurred **on the device, before any write to storage.**
- No driver identity is collected.
- First and last 200 m of every trip discarded, so home and depot stay private.
- Vectors, not images, leave the vehicle — about 10 KB per detection.

8.8 Deployment phases

Each phase is independently useful; nothing waits on a later one.

PHASE	SCOPE	DELIVERS
1	5 trucks, one ward	Detection and a crack inventory
2	Three-week baseline	Growth rates; the first sealing schedule
3	Full fleet	Pre-monsoon campaign and repair verification
4	Multi-ward	Platform, second sensing module

Fifty questions, answered

Grouped by where they come from. The hostile ones are here too.

On the image processing

Q1 Why not deep learning? Everyone else is using YOLO.

Because our contribution is the *measurement over time*, not the detection. A detector returns a bounding box; we need a traced length in millimetres to subtract between passes. Classical methods give that, are auditable line by line, and need no training data we have not earned. A TFLite detector on RDD2022 is the production path and it feeds this same geometry stage.

Q2 How exactly do you find the crack?

Downscale to 760 px, convert to greyscale on BT.601 weights, sweep a darkness threshold from 70 to 180 keeping the setting with the best contrast $\times$ reach, apply a 3 $\times$ 3 morphological opening to kill aggregate speckle, label 8-connected components, and score each on four shape gates. Half the winner's perimeter is the traced length.

Q3 Why is arc length half the perimeter?

For a long thin shape the boundary runs up one side and back down the other, so the perimeter is about twice the length. It follows every bend, which a straight-line ruler between endpoints would not — and a crack that wanders is genuinely longer than the distance across it.

Q4 What stops a shadow being measured as a crack?

Four gates and a contrast term. A shadow is usually too large (area fraction $\geq 6\%$), runs off the frame edge ($\times 0.18$ penalty), or is not much darker than the scene mean, which demotes it in the contrast $\times$ span ordering. It is not infallible, which is why the operator can cycle to the next candidate.

Q5 Why 760 pixels?

Wide enough that a 3 mm crack at 1.2 m is still 2–3 px across; small enough to run twelve threshold passes in well under a second in a browser. It is stored with every reading, because changing it changes the millimetres.

Q6 Why dilate then erode, not the other way round?

Dilate-then-erode is an *opening*: it removes small isolated objects and keeps large connected ones — exactly right for killing aggregate speckle while keeping a crack. The reverse, a closing, would fill gaps and keep the speckle.

Q7 Why 8-connectivity?

A crack at 30° to the pixel grid is a staircase of diagonal steps. 4-connectivity would break it into fragments, each failing the size gate.

Q8 What is P^2/A and why use it?

Thinness — the isoperimetric ratio. Minimum $4\pi \approx 12.6$ for a circle, large for a long thin line. It is scale-invariant, so the same crack at 1 m and at 1.5 m scores alike.

Q9 Your threshold is fixed. Roads are not.

It is not fixed. Twelve thresholds are tried, 70 to 180, and the one whose winner has the highest contrast $\times$ reach is kept. That is why the app works on the demo images despite their being rendered at exposures spanning 28 grey levels.

Q10 What about a tar patch or old sealant?

Genuine false positive, and we say so. Geometry cannot separate it — it is long, thin and dark. Time can: a sealant line is present on day one and never grows. That persistence rule needs exactly the repeat-visit data this system collects, which is why the answer is in the architecture rather than in a better filter.

Q11 What if it picks the wrong shape?

"Wrong shape? Next candidate" cycles through the runners-up. And when nothing crack-shaped was found at all, the interface says so rather than presenting the darkest blob as a result.

On GPS and matching

Q12 GPS is only good to 5–10 m. How is this the same crack?

It is not, on GPS alone, and the app never claims otherwise. Three signals vote: distance within 15 m, heading within 45°, and a 64-bit scene fingerprint within 12 bits. Position and heading together, or an unmistakable scene on its own. Every match prints its own reasoning on screen.

Q13 Why 15 m when the error is 5 m?

Because the tolerance must exceed the measurement error or real revisits get rejected. Two fixes each good to ± 5 m can differ by 10 m while describing the same point. The heading and appearance terms carry the discrimination the loosened radius gives up.

Q14 What if there is no GPS in the photo at all?

Three fallbacks in order: the browser's geolocation, a manually typed fix, or the scene fingerprint alone — which at 7 bits or fewer is strong enough to carry the decision by itself. And the position is then written into the file's EXIF as it is stored, so the photograph carries it from then on.

Q15 How does the image fingerprint survive different lighting?

It is a *difference* hash: each bit records whether one coarse patch is brighter than the patch to its right. Add 20 to every pixel and every comparison gives the same answer. Measured across our eighteen passes at deliberately varied exposure: never more than 2 bits of 64 different.

Q16 Could two different roads hash the same?

64 bits is 1.8×10^{19} possibilities, and the hash is only ever compared against spots already within 15 m. A collision would have to be a different crack in the same 15 m circle with the same coarse layout — and the distance and heading terms still have to agree.

Q17 Two cracks one metre apart. What then?

The honest answer: no sensor in this set separates them, because they are inside every tolerance we have. So the person holding the camera does — a "this is a different crack" control opens a new spot. Pretending an algorithm could do it would be the dishonest answer.

Q18 What if the position matches but the picture does not?

It is flagged, not silently accepted: "The position matches but the scene does not." Usually it is a wet road. Sometimes it is a different crack two metres over. The system does not know which, so it says so and offers the override.

Q19 Does accuracy improve over time?

Yes. Each pass re-centres the spot on the mean of its own fixes, and on the *circular* mean for heading. Standard error of the mean falls as $1/\sqrt{n}$, so an eighteen-pass spot is about four times better located than a single fix.

Q20 Two trucks photograph the same crack.

Deduplicated by position, heading and visual match — the same three-way vote. It is counted once however many vehicles pass it.

On measurement

Q21 How do we trust your millimetres?

You do not have to, and that is the point. A traced outline adds its own width — a constant bias b . Every reading is $L+b$. The growth is $(L_2+b) - (L_1+b) = L_2 - L_1$: **the bias cancels exactly**. Growth is what we report; absolute length is not the claim.

Q22 Where does mm-per-pixel come from with no ruler?

The pinhole relation. A 35 mm-equivalent focal length is defined against a 36 mm frame, so ground width = $36 \cdot d/f$ and one pixel is that over the pixel width. For our run: $36 \times 1200 \div (80 \times 760) = 0.7105 \text{ mm/px}$.

Q23 What if the photo has no lens data?

A 26 mm phone lens is assumed and **every reading it produces says "assumed" on its face**. It is rung four of four, used only when nothing else is known. Because growth is a subtraction, a wrong constant scales both readings alike and largely cancels out of the number we sell.

Q24 What is your millimetre accuracy?

We report the measured scatter rather than assert a figure: RMSE 1.18 mm about the fitted line over eighteen readings, largest single residual 2.57 mm. The training tab records trials against a physical ruler; that number is empty until somebody does them, deliberately.

Q25 Distance varies between passes. Doesn't that ruin it?

It is the dominant remaining error — ± 10 cm at 1.2 m is about $\pm 8\%$ of scale — and it is why the truck deployment uses a rigid mount at a fixed height. In the prototype it shows up as scatter about the fit, which the R^2 and the confidence interval expose rather than hide.

On the statistics

Q26 Why a straight line? Crack growth is not linear.

Agreed — over a season it is not. Over a two-to-three-week window it is a good local approximation, and R^2 0.974 over eighteen readings says the data does not contradict it here. A linear fit is also the honest choice with this much data: fitting a curve to eighteen noisy points would produce confident nonsense.

Q27 How confident is 1.392 mm/day?

Standard error 0.057 mm/day; 95% interval 1.271 to 1.513. Propagated to the crossing, that is 10.4 to 12.4 days against a point estimate of 11.3 — **about ± 1 day**, which is far inside what a fortnightly maintenance decision needs.

Q28 Two points give $R^2 = 1$. Isn't that misleading?

It would be, so the reading count is always printed beside R^2 . " R^2 1.00 over 2 readings" reads differently from " R^2 0.97 over 18". Below 0.6 the app also prints a warning and calls the date rough.

Q29 What is the minimum number of photographs?

Two, on different days, gives a rate. The deck's three-week baseline is what makes the rate *trustworthy*. One photograph gives no rate at all, and the app shows grey rather than green for it.

Q30 What if a reading is an outlier?

It shows as a large residual and depresses R^2 . The gallery lets you remove that photograph, after which the rate, the date and the seal list recalculate from what is left. We do not auto-reject outliers — silently discarding data the operator has not seen is how a measurement system starts lying.

Q31 What if the crack shrinks?

Slope ≤ 0 gives STABLE, a green light, and no date. It is what a sealed or a mis-measured crack looks like, and the verification box distinguishes the two by asking whether the last pass broke the trend.

On the formula

Q32 Where does the rainfall number come from?

A bundled table of monthly normals for 47 Indian stations, apportioned across the fourteen-day window, nearest station with its distance reported. The API labels it `basis: "normal"` and never says forecast. Post a real IMD figure and it is used instead.

Q33 Why divide rainfall by 60?

60 mm is an ordinary wet fortnight over most of the country, so it puts the factor near 2 in a normal monsoon. It is a chosen constant, returned in `/api/risk` under `assumptions`, and the first thing to re-fit against a season of measured growth.

Q34 Your traffic numbers are made up.

They are class defaults, and every answer says `assumed` or `counted`. A municipality with real counts enters them and the term changes in front of you — it is a live control on the report tab.

Q35 Pavement engineering uses a fourth-power law. Why square root?

The fourth power describes *structural damage from axle load*. Applied here it would put a highway 200× above a lane and drown the measured term — the ranking would become a road-class sort. We are ranking how much sooner an already-growing crack needs sealing, so a gentler curve keeps the measurement in charge. Stated as an assumption, not smuggled in.

Q36 Why is the score logarithmic?

Because the product spans orders of magnitude. We built it linear first and watched four genuinely different spots all score 100 — at which point the ranking stops ranking. Log between 0.05 and 10 mm/day effective discriminates across the whole realistic range.

Q37 Does rain change your measured growth rate?

No, and that separation is deliberate. `mm_per_day` stays exactly what the readings said. The rain produces a *second* crossing date, printed beside the dry one, and the difference between them — six days on our run — is what sealing before the monsoon is worth.

Q38 Why multiply rather than add the terms?

Because the effects compound physically: rain accelerates a growing crack in proportion to how fast it is already growing; traffic works open a crack that water has already undermined. Addition would let a stationary crack on a highway outrank a fast-growing one on a lane, which is exactly the wrong answer.

On the system and deployment

Q39 Will this work at 25 km/h from a moving truck?

6.9 m/s with a 1/1000 s shutter is 6.9 mm of travel — a few pixels of smear at our ground resolution. That is why the slow fleet was chosen deliberately. At 60 km/h the same shutter gives 17 mm and the measurement stops meaning anything.

Q40 How much data does a truck send?

About 10 KB per detection — a polyline and a class, not an image. Two hundred defects a day is roughly 2 MB, over depot Wi-Fi at night. No mobile data, no continuous video anywhere in the design.

Q41 Won't the phone overheat?

Screen off, 1–2 FPS, vectors rather than video, no continuous recording. The compute load is low by design, and the gating stage rejects most frames before any detection runs.

Q42 What about privacy and the DPDP Act?

Faces and plates blurred on the device before anything is written to storage. No driver identity collected. First and last 200 m of every trip discarded so home and depot stay private. Vectors, not images, leave the vehicle.

Q43 What happens when the road is resurfaced?

Resurfacing is detected as a wholesale change in the scene fingerprint across every spot on that stretch, and the segment baseline resets. A new surface is never logged as a failed repair.

Q44 How do you verify a repair actually happened?

Tomorrow's collection round drives over yesterday's seal. In this build, with no repair record, we report the honest version: whether the last pass continued the trend or broke it. A length that stops growing after a crew visited is what a seal looks like in the readings.

Q45 Why SQLite? Real systems use PostgreSQL.

They do, and the production design says PostGIS. This prototype must run on a municipal laptop with no internet and no admin rights — `python3 raahi.py` and nothing else. The schema migrates unchanged when spatial indexing over a city starts to matter.

Q46 How do you know the app is correct?

68 end-to-end checks in `tools/selftest.py`. It starts a real server, writes real geotagged photographs, uploads them over HTTP and asserts what comes back. Nothing is mocked; the detector inside the test is the browser's own, reimplemented in Python.

The hostile ones

Q47 What is your detection accuracy? Everyone else quotes mAP.

We have not measured one, so we do not quote one. The detection quality page is written so that it *cannot* print a headline figure without a named test split and a stated sample size. This build measures growth; that is the claim, and it is a strength stated plainly rather than an absence dressed up as a table.

Q48 Isn't this just crack detection with extra steps?

Crack detection answers "is there a crack here". This answers "which crack, and by when" — and the second question is the one a maintenance budget is spent against. Detection is a solved problem we did not solve; the measurement over time is the part nobody has, because nobody else returns to the same metre daily.

Q49 Your demo images are synthetic. Does it work on real roads?

The synthetic set exists so the loop can be demonstrated indoors in thirty seconds, and the app cannot tell the difference — it reads their EXIF and matches them like any photograph. Real phone photographs work through the same path; the untested part is not the software but the field conditions, which is why daylight and close range are stated limits rather than assumed ones.

Q50 What is the single biggest weakness?

That the growth measurement depends on a consistent camera geometry between passes, and the prototype achieves that by asking a person to stand in roughly the same place. The rigid truck mount is what makes it robust, and that is exactly the part we have designed but not built. We would rather you test that limit than discover one we hid.

Every figure in this document comes from the run committed at `docs/sample-run/` in the repository: eighteen photographs, one crack, 1.392 ± 0.057 mm/day at R^2 0.974. The input images and the raw JSON output are both there.

RAAHI · Team Innovators_SIH26 · Smart India Hackathon 2026 · SIH26205 · Transportation & Logistics